Feasibility Analysis and Requirements Specification

Agentic University Student Services Portal

Group	Group ID: _____
Members	1. _____ 2. _____ 3. _____ 4. _____
Course	IT Project I and II (BC362916 + BC362914)
Instructor	Dr. Aftab Alam Janisar
Milestone	Week 05: Feasibility Analysis and Requirements Engineering
Date	2 October 2026

Note on naming: the target institution in this report is a fictional mid-sized university, referred to throughout as **the University**. No real institution is described, and no real institutional data is used.

1. Problem Summary and Target Domain

1.1 Problem Restatement

A university website today is a publishing platform. It presents program pages, announcements, downloadable handbooks and a contact address. It serves the institution's need to publish, not the student's need to act. Three observable failures follow from that design.

The website accepts no action. A student can read the transcript request procedure, yet cannot submit the request, check whether it is ready, or book the advisor meeting the procedure tells them to book. The same applies to every scarce resource the campus owns: a study room, an instrument room, a club activity room, a seat at a workshop. The site ends exactly where the student's actual task begins, so every task it describes turns into an email, a phone call, or a queue at a counter.

Published information is not answerable information. Regulations, fee rules and deadline changes sit inside PDF handbooks and dated announcement posts, and site search matches page titles rather than the clause inside the file. A student must already know which document holds the answer before they can find it, which makes the official source technically available and practically unreachable. Students therefore bypass it and ask a senior or a classmate, so unofficial answers spread faster than official ones.

Service continuity breaks at the staff boundary. Student services keep no shared case record, so a student's context lives only in the memory of the staff member who handled them. When that person is absent the next day, a student pursuing a multi-step request such as visa documents restates the entire case from the beginning, and the replacement staff member repeats the same search and sometimes gives a different answer. Continuity of service therefore depends on individual staff attendance, which no reliable service can be built on.

1.2 Project Objectives

1. Build an indexed regulation corpus and a retrieval agent that answers at least 80 percent of a 50-question benchmark correctly and cites the source section for every answer.

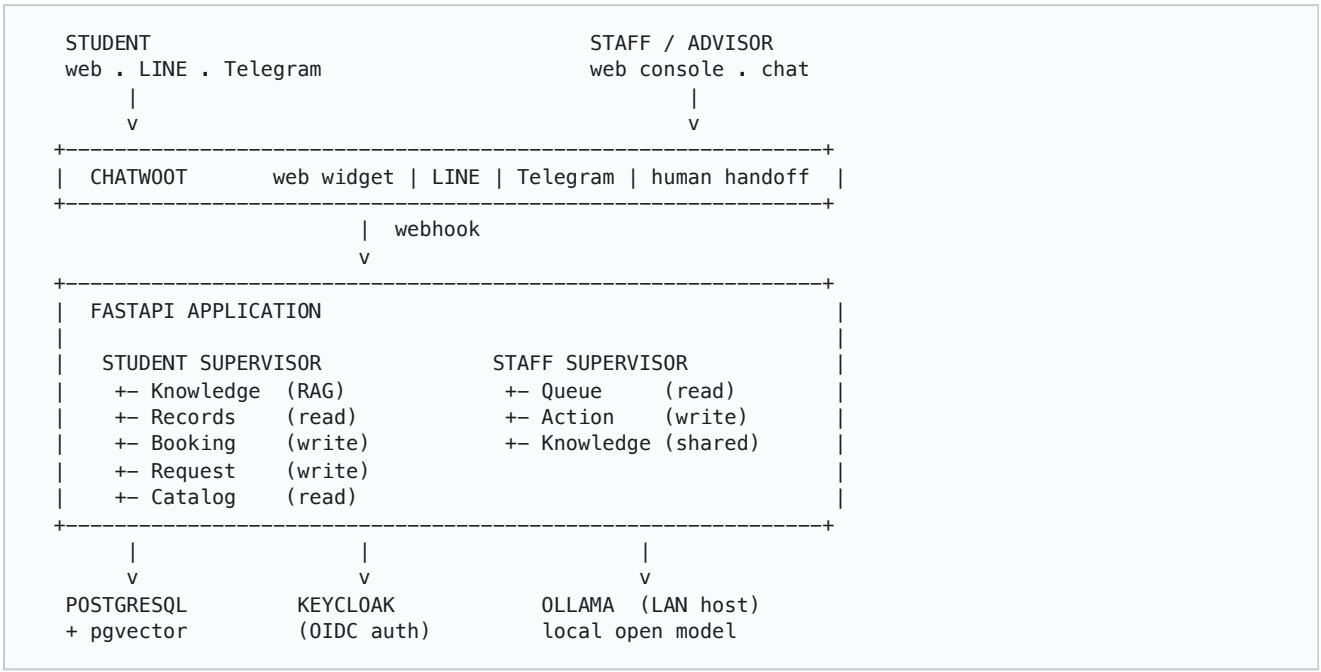
2. Move the student's common tasks inside the website: ask a policy question, read their own record, book any campus resource, and submit and track any service request.
3. Give every booking and every request a durable state and a visible history, so that no student travels to campus to ask about progress.
4. Preserve conversation context across staff changes and across channels, so that a student never restates a case the institution already holds.
5. Give staff the same conversational interface over their own queues, so that routine administration does not require learning a new console.
6. Keep every component free and open source and run the language model locally, so that the prototype costs nothing to operate.

1.3 Target Domain and User Profile

The operational environment is the student services function of the University, a mid-sized bilingual campus operating in English and Thai. The environment consists of a student services office of roughly three to six staff, academic advisors who hold weekly office hours, a facilities team that owns bookable rooms and equipment, a library, and a student body that already communicates through chat applications on mobile phones. The office works standard weekday hours. Demand is not flat, and it concentrates sharply around registration, fee deadlines and result publication.

User group	Technical literacy	Device and context	What they need from the system
Students (highest volume)	High for chat applications, low for institutional software. They will not read a manual.	Mobile phone, often outside office hours, intermittent network. English or Thai.	A correct answer immediately, and the ability to finish the task in the same conversation.
Student services staff	Comfortable with office software and a shared inbox. Not developers.	Desktop at a counter during working hours.	Fewer repeat questions, a queue they can see, and the full history of any case they pick up.
Advisors and facility owners	Moderate. They want to spend no time administering a calendar.	Laptop, between teaching commitments.	Published availability that accepts exactly one booking per slot.
Administrator	Technical. Runs accounts and the deployment.	Desktop with container access.	Account control, an audit trail, and one place to update the document corpus.

1.4 System Context



Retrieval-augmented generation (RAG) means the model answers only from text the system retrieves first. OpenID Connect (OIDC) is the standard protocol Keycloak uses to issue and verify login tokens. Ollama is an open-source runtime that serves an open-weight language model over HTTP.

2. Multi-Dimensional Feasibility Analysis

2.1 Technical Feasibility

Chosen language, frameworks and APIs

Layer	Chosen technology	Licence and cost	Reason for the choice
Backend language	Python 3.12	PSF, free	The agent and retrieval ecosystem is strongest in Python, and it is the team's primary language.
Web framework	FastAPI with Uvicorn	MIT, free	Native asynchronous support, which matters because every agent turn waits on a model call.
Agent orchestration	LangGraph with langgraph-supervisor	MIT, free	Gives an explicit state graph and a supervisor router instead of hidden control flow. Two supervisors run in one process with different tool permissions.
Language model runtime	Ollama, serving an open-weight model over the local network	MIT, free	No API key, no quota and no recurring cost, which satisfies the zero-cost constraint outright.
Database	PostgreSQL 16 with the pgvector extension	PostgreSQL licence, free	One engine holds both the transactional records and the document vectors, which removes a second data store.

Authentication	Keycloak	Apache 2.0, free	Removes the need to write any credential or token handling code.
Messaging and handoff	Chatwoot, self-hosted	MIT, free	Provides the web widget, the LINE connector, the Telegram connector and the human takeover interface in one service.
Frontend	React with Vite, one application with routes gated by Keycloak role	MIT, free	Public pages, student portal and staff console ship as one build, which suits a four-person team.
Deployment	Docker Compose, with an ngrok tunnel for the LINE and Telegram webhooks	Free tiers	One command starts the whole stack, which makes the demonstration repeatable.
Testing	pytest, with Locust for concurrency	MIT, free	Needed for the access-control negative tests in NFR-02 and the booking load test in FR-09.

Architecture realism for an undergraduate prototype

The architecture is one application process, one database, one identity service, one messaging service and one model runtime. There is no message broker, no microservice boundary, no orchestration platform and no service mesh. The multi-agent part is two routing graphs inside a single process, so a failure in one specialist agent does not require distributed debugging.

The design decision that makes a wide feature list affordable is that features share engines rather than multiply them. Every bookable thing on campus is one *resource* row with a slot table, so an advisor appointment, a study room, an instrument room, a club activity room and a workshop seat all use a single reservation path with one locking strategy. Every trackable thing is one *request* row with a type and a state, so a transcript request, a library hold and a maintenance report all use a single state machine with one notification path. Two engines therefore cover eight user-visible features, and the Booking agent and the Request agent are each written once.

Model access, paywalls and rate limits

The prototype has no external paid dependency and no rate limit, because the language model runs on hardware the team owns. One member hosts Ollama and binds it to the local network, and the other three point their application configuration at that host. If the campus network blocks machine-to-machine traffic, the team uses the free tier of a mesh network service such as Tailscale, which makes the same address reachable from any network without a configuration change.

Production note. Local inference is the correct choice for a zero-budget prototype and is not the correct choice for a deployed service. A real deployment serving a full student body would need either a dedicated inference server with a graphics processor, or a paid hosted model through a provider such as OpenRouter or a direct vendor API, at a recurring cost per million tokens. The application calls the model through a single adapter module with one interface, so moving from Ollama to a paid provider means changing configuration and one file rather than rewriting the agents. This report treats that migration as a known production cost, not as a hidden assumption.

Technical skill gaps

Skill	Team level	Evidence	Gap and mitigation

Python, FastAPI, PostgreSQL	Working	A team member has built and shipped a production backend on this stack.	No gap. That member owns the backend track and reviews the others' work.
LangGraph supervisor multi-agent	Working	Prior migration of a single agent to a supervisor with specialist agents in a live system, including a staff-side supervisor.	No gap in the pattern. The other three members need two weeks of ramp-up, scheduled in W1 and W2.
Retrieval-augmented generation	Working	Prior document-grounded question-answering agent.	No gap for English. See the Thai gap below.
Keycloak and OIDC	Working	Prior Keycloak integration including role mapping.	No gap.
Chatwoot, LINE and Telegram connectors	Partial	Prior Chatwoot integration with inbound routing, bot standdown and agent takeover.	The handoff model is known. LINE needs a LINE Developers account and a channel access token, which is free but takes setup time. Scheduled in W7, with Telegram built first as the simpler connector.
Local model quality and tool calling	**Gap**	Prior work used hosted models, which follow tool-calling instructions more reliably than small local models.	**Highest risk gap.** Mitigation: a W2 bake-off measures two or three candidate models against the team's own English and Thai benchmark before any agent is written. See risk R1.
Formal evaluation of agent quality	**Gap**	Previous work was verified by hand, never scored against a benchmark.	Mitigation: the benchmark is written in W1 and W2, before agent code, and scored by two independent raters.
Thai-language retrieval	Partial	No prior Thai retrieval work by the team.	Mitigation: use a multilingual embedding model, keep an aligned English and Thai corpus, and include Thai questions in the benchmark from W2 so that quality is measured rather than assumed.
Load and concurrency testing	Partial	The team has written transactional locking but never a formal concurrent load test.	Mitigation: Locust, an open-source load tool. One day of study is sufficient, scheduled alongside the booking engine.

2.2 Operational and Resource Feasibility

Fit with actual user workflows and adoption

Students already answer questions through chat on a phone, so the student-facing interaction adds no new habit, and LINE in particular is a channel this user group opens daily. Adoption risk sits with staff, not with students, for two reasons. First, an assistant that answers questions removes work that staff currently own, which some staff read as a threat rather than as relief. Second, a staff console is a new tool in a workflow that currently runs on a shared inbox.

The design answers both. The assistant never decides anything: it escalates on request and on low confidence, and a human owns every exception. The staff-side supervisor means a staff member can work through the same chat interface they already use rather than learning a console, so the console becomes an

option rather than a requirement. Training for a staff member is a single page describing three actions: take over a conversation, change a request state, and release the conversation. Students need no training.

Team and role allocation

Track	Owner	Responsibility	Interface to the other tracks
Infrastructure and identity	Member 1	Docker Compose, PostgreSQL schema and migrations, Keycloak realm and roles, the Ollama host, continuous integration.	Publishes the database schema and the token claims that every other track depends on.
Retrieval and evaluation	Member 2	Corpus authoring in English and Thai, indexing, embedding choice, the benchmark, and all measurement.	Publishes a retrieval function with a fixed signature, and owns the quality numbers reported in NFR-05.
Agents and tools	Member 3	Both supervisors, the five student-side agents, the three staff-side agents, and every tool function.	Consumes the schema and the retrieval function. Publishes the chat endpoint.
Frontend and channels	Member 4	The React application, public pages, student portal, staff console, Chatwoot configuration, LINE and Telegram connectors.	Consumes the chat endpoint and the REST endpoints. Owns the usability measurement in NFR-03.

The split is by layer rather than by feature, because four people editing the same agent file produce merge conflicts, while four people working behind published interfaces do not. Each interface is agreed in W1 and frozen, so a late track never blocks an early one.

Hardware, tooling and budget

Resource	Source	Cost
All server and client software	Open-source images from Docker Hub	0
Language model inference	Ollama on team hardware, open-weight model	0
Development machines	Two laptops with 16 GB of memory, two with 8 GB	0
Public webhook endpoint for LINE and Telegram	ngrok free tier	0
Cross-network access to the model host, if required	Tailscale free tier	0
Source control and issue tracking	GitHub free plan	0
Document corpus and all records	Authored and generated by the team	0
Total monetary budget for Project I		**0**

The binding resource is memory, not money, and the team's machines are unequal. The allocation below resolves that.

Machine	Role	Approximate memory use
16 GB laptop, member 1	**Model host.** Runs Ollama only, serving the other three over the local network.	Operating system 4 GB, model runtime 6 GB, headroom 6 GB.

16 GB laptop, member 3	**Integration host.** Runs the full container stack and is the demonstration machine.	Operating system 4 GB, Chatwoot 2.5 GB, Keycloak 0.7 GB, PostgreSQL 0.5 GB, application 0.5 GB.
8 GB laptops, members 2 and 4	**Development only.** Run PostgreSQL and the application locally and point at the shared Keycloak, Chatwoot and Ollama instances.	Operating system 4 GB, PostgreSQL 0.5 GB, application 0.5 GB.

No member runs the model and the full container stack on the same machine, which is the configuration that would exhaust 16 GB. The figures above are estimates and are verified by measurement in W2, before any agent work begins. Risk R5 records the reduced configuration if the measurement proves them wrong.

2.3 Time Feasibility

The team has no other project this semester and has four members, so capacity is higher than a single-developer plan assumes. The schedule below is written in relative weeks from the start of implementation, labelled W1 onward, so that it stays correct whatever the final submission date turns out to be. It assumes ten working weeks for Project I. The four tracks run in parallel, which is why more features fit than a serial plan would allow.

Week	Infrastructure	Retrieval and evaluation	Agents and tools	Frontend and channels
W1	Compose stack, schema, Keycloak realm, interfaces frozen	Corpus sourcing, English drafting	Ramp-up on LangGraph, tool signatures agreed	React skeleton, routing, Keycloak login
W2	Ollama host on the network, memory measured	**Benchmark written, model bake-off run**	Supervisor skeleton with one stub agent	Public site pages
W3	Seed data generator, audit table	Indexing, embedding choice, Thai corpus	Knowledge agent end to end	Chat widget embedded, first slice visible
W4	Resource and slot schema	Retrieval tuning against the benchmark	Records agent, student supervisor routing	Student portal views
W5	Transaction and locking review	Routing set labelled and scored	**Booking agent with slot locking**	Booking views, availability display
W6	Notification outbox	Thai benchmark scored	**Request agent and state machine**	Request views and history display
W7	Webhook routing through ngrok	Corpus freeze for evaluation	Catalog agent and library holds	**Chatwoot, then Telegram, then LINE**
W8	Role and permission matrix	Rater training, first scoring pass	**Staff supervisor: Queue and Action agents**	Staff console
W9	Access-control negative suite, load test	Final scoring with two raters	Bulk-action caps and confirmation steps	Usability test with five students
W10	**Buffer. No new features. Defect fixing, documentation and demonstration rehearsal.**			

Two of the ten weeks carry no new functionality: W2 produces measurement instead of features, and W10 is reserved for defects. If the schedule slips, the cut order is fixed in advance and the team cuts from the bottom of this list: the LINE channel, then the staff console interface, then the Catalog agent, then the staff-side supervisor. The Knowledge agent, the Records agent, the Booking agent and the student supervisor are the irreducible core, and they are scheduled earliest for that reason.

Deferred to Project II

- Cloud deployment with a real domain name, managed TLS certificates and a paid inference provider.
- Course add and drop requests with prerequisite and timetable-clash checking.
- Payment processing and payment slip reading.
- Integration with a real student information system.
- Full-text retrieval inside library holdings, which requires licensed content.
- Native mobile applications, voice input, and languages beyond English and Thai.

2.4 Data Feasibility and Hardware Access

No real institutional data is used, and none is required. The project has no partner university and therefore no access to real student records, and seeking such access would raise a privacy question that an undergraduate project cannot properly answer. Every data set is constructed instead.

- **The regulation corpus** is authored by the team from publicly published university handbooks, academic regulations and fee schedules, adapted into the fictional University and produced in aligned English and Thai versions. The corpus is realistic in structure, length and ambiguity, which is what the retrieval problem depends on. It is committed to the repository, so every evaluation run is reproducible.
- **Student records** are generated: synthetic names, generated identifiers, synthetic timetables, registration states and fee balances. No record describes a real person.
- **Bookable resources and the library catalog** are generated: rooms, instruments, advisor office hours, workshop seats, and catalog entries with invented titles and availability. No copyrighted text is stored.
- **The evaluation sets** are 50 graded benchmark questions in English and Thai and 100 labelled routing messages, written by the team in W1 and W2 and frozen before agent development begins.

Privacy and security factors. Even with synthetic data, the system is built as though the data were real, because the access-control design is the part that must be correct. Every agent tool takes the subject identifier from the verified login token and never from the message text, so a user cannot ask the assistant to read another person's record. The staff-side supervisor holds a separate, wider permission set that is unreachable from a student session, and every staff write action is capped and recorded. Conversation transcripts are stored because the handoff feature requires them, and personal identifiers are excluded from application logs. All browser traffic uses TLS.

Hardware access. Four laptops owned by the team, two with 16 GB of memory and two with 8 GB, allocated as described in Section 2.2. No graphics processor is available, which is the reason the W2 bake-off is limited to models that run acceptably on unified memory. No special hardware, camera or payment terminal is involved.

2.5 Risk Identification and Mitigation

ID	Risk	Likelihood / Impact	Mitigation

R1	The local model is not reliable enough at tool calling, so agents select the wrong tool or produce malformed arguments.	High / High	The W2 bake-off tests candidate models on tool calling before any agent is written, not after. Tool schemas stay small and flat. A validation layer rejects malformed arguments and retries once with a corrective message. If no local model reaches the threshold, the adapter switches to a hosted provider and the report states that limitation.
R2	Thai retrieval or Thai generation quality falls far below English quality.	Medium / Medium	Thai questions are in the benchmark from W2, so the gap is measured early rather than discovered late. The corpus is aligned English and Thai. If Thai generation stays weak, the Thai path returns the retrieved Thai passage directly with its citation and offers a human handoff.
R3	Prompt injection: text inside a retrieved document or a user message instructs the agent to act or to reveal another user's data.	Medium / High	Tools take the subject from the token, never from text. Each agent holds an explicit tool allowlist, and the staff tool set is unreachable from a student session. No tool accepts free-form database queries. A negative test suite attempts cross-account access on every tool and runs in continuous integration.
R4	Scope creep. The feature list is wide, and a university offers an endless supply of plausible additions.	High / High	Section 6 freezes the in-scope list. Any addition requires removing something of equal size. The shared-engine design means a new feature is accepted only if it reuses the booking engine or the request engine and needs no new one.
R5	Memory exhaustion on the 8 GB machines or on the integration host.	Medium / Medium	Memory is measured in W2 before agent work. The 8 GB machines run only PostgreSQL and the application and share the heavy services. The documented fallback removes Chatwoot from a local machine and serves the web widget directly, which costs LINE and Telegram on that machine but not on the integration host.
R6	The campus network blocks machine-to-machine traffic, so the three client machines cannot reach the Ollama host.	Medium / Medium	Tested in W2, not on demonstration day. The first fallback is the free tier of a mesh network service, which makes the model host reachable from any network. The second fallback is a phone hotspot shared between the four machines.
R7	Integration failure between four parallel tracks, where one track's late delivery blocks the others.	Medium / High	Four interfaces are agreed and frozen in W1: the database schema, the token claims, the retrieval function signature and the chat endpoint. Each track codes against a stub of the others from W1, so no track waits for a real implementation.
R8	The agent gives a confident answer that the source documents do not support.	Medium / High	Every answer must carry a citation, and an answer with no retrieved passage above the similarity threshold is refused rather than generated. NFR-05 measures this directly.
R9	The LINE channel needs a LINE Developers account and setup that takes longer than expected.	Low / Low	The account is registered in W1, six weeks before the connector is needed. Telegram is built first and proves the multi-channel architecture independently, so LINE is never on the critical path.

3. Feasibility Recommendation

PROCEED WITH ADJUSTMENTS

The project is feasible for a four-person team within Project I, with the adjustments listed below. The architecture is one application process, one database and three supporting services, and one team member has already shipped production work on every component of it, so the technical risk sits in model quality rather than in infrastructure. Every tool is free and open source and the language model runs on hardware the team already owns, so the total monetary budget is zero without compromise. The feature list is wide but not deep, because eight user-visible features share two engines, one reservation engine with slot locking and one request engine with a state machine, each written once. Three adjustments keep the schedule honest: the model is chosen by a measured bake-off in Week 2 rather than assumed, the project commits to English and Thai only with Thai measured from the start rather than promised, and every feature that would need a new engine, including payments, timetable-clash checking and full-text library search, moves to Project II. The two genuine gaps are local-model tool-calling reliability, mitigated by testing it before any agent is written, and formal evaluation, mitigated by writing the benchmark before the agents. With those adjustments the project is ready to move to requirements and system architecture.

4. Stakeholders and Requirements Specification

4.1 Stakeholder Register

Class	Stakeholder	Interest in the system	Influence
Primary users	Enrolled students	Get a correct answer immediately and finish a task without visiting a counter.	High. Non-adoption makes the system pointless.
	Student services staff	Handle fewer repeat questions, work their queue through chat or console, and pick up any case with its full history.	High. They can refuse to use the system.
Secondary users	Academic advisors	Publish availability once and receive exactly one booking per slot.	Medium. They own the appointment data.
	Facility and library staff	Room, equipment and catalog bookings arrive as records rather than as phone calls.	Medium. They own the resource data.
	Head of student services	See request volume and turnaround that spreadsheets cannot produce.	Medium. Sponsors the deployment decision.
System administrators	IT administrator	Manage accounts and roles, run the container stack and the model host, keep the corpus current.	High. Controls deployment, access and account provisioning.

External systems and actors	Ollama model host	Generates every agent reply. Internal to the team but external to the application.	High. If it is unreachable, no agent replies.
	LINE Messaging API and Telegram Bot API	Deliver and receive messages on the two external channels.	Low. The web widget works without either.
	Prospective students and external verifiers	Affected by public answer quality but do not operate the system.	Low.

4.2 Functional Requirements

The requirements use the terms *bookable resource* and *service request* deliberately. A bookable resource is an advisor office hour, a study room, an instrument room, a club activity room or a workshop seat. A service request is a document request, a library hold or a maintenance report. One requirement therefore covers every instance of its type, which is also how the code is written.

ID	Requirement	How it is tested
FR–01	The system shall authenticate every user through Keycloak using OpenID Connect, and shall reject with HTTP 401 any request to a protected endpoint that carries no valid access token.	Automated test sends a request with no token, an expired token and a valid token.
FR–02	The system shall assign each account exactly one of the roles student, staff, advisor or administrator, and shall return HTTP 403 when an account calls an endpoint its role does not permit.	Automated test calls every endpoint once per role.
FR–03	The system shall route every student message to exactly one specialist agent among Knowledge, Records, Booking, Request and Catalog, and shall store the chosen route with the message identifier.	Query the routing log after the 100-message labelled set runs.
FR–04	The system shall answer a policy question only from passages retrieved from the indexed corpus, and shall display the source document title and section with every answer.	Inspect all 50 benchmark answers for a citation. Zero answers without one.
FR–05	The system shall refuse to answer and shall offer a human handoff when no retrieved passage scores above the configured similarity threshold.	Submit 10 questions with no corpus coverage and confirm all 10 refuse.
FR–06	The system shall accept student messages in English and in Thai, and shall reply in the language of the incoming message.	Submit 20 English and 20 Thai questions and confirm the reply language matches in all 40.
FR–07	The system shall return the signed-in student's own timetable, registration status and outstanding fee balance on request, and shall refuse any request that names another student.	Negative test attempts retrieval of 30 other-student records and expects 30 refusals.
FR–08	The system shall list the available slots of any bookable resource for a date range given by the student.	Compare the returned list against the database for 10 date ranges across 3 resource types.

FR–09	The system shall reserve a selected slot of a bookable resource inside a single database transaction, and shall reject any second reservation of a slot already taken.	200-request concurrent load test on one slot expects exactly one success and zero double-bookings.
FR–10	The system shall allow a student to cancel or reschedule only their own reservation, and only before the cut-off time configured for that resource type.	Test cancellation before the cut-off, after the cut-off, and on another student's reservation.
FR–11	The system shall accept a service request of a configured type, shall store it in exactly one of the states submitted, under review, ready or closed, and shall record a timestamped history entry for every state change.	Create one request of each type, move it through all four states and read back the history.
FR–12	The system shall notify the requesting student on their originating channel within 60 seconds of a change to the state of their reservation or request.	Change a state and measure delivery time on the web widget, on LINE and on Telegram.
FR–13	The system shall hand a conversation to a human staff member on student request or on low routing confidence, and shall generate no further automatic reply until the staff member releases it.	Trigger a handoff and confirm the agent stays silent for 10 subsequent messages.
FR–14	The system shall exchange messages on the web widget, LINE and Telegram through Chatwoot, and shall present one continuous conversation history for a student across all three channels.	Start a conversation on the web, continue it on LINE and on Telegram, and read back a single history.
FR–15	The system shall provide a staff-side agent that lists a staff member's open queues and performs state changes and cancellations on request, shall require an explicit confirmation before any action affecting more than one record, and shall write an audit record containing actor, action, target and timestamp for every write operation performed by any agent.	Perform one single action and one bulk action as staff, confirm the bulk action prompts first, then read both audit rows.

4.3 Non-Functional Requirements

The performance targets below are set for a locally hosted open-weight model on consumer hardware, which is slower than a hosted API. The system streams its reply, so the first visible token matters more to the user than the completion time, and both are stated.

ID	Quality attribute	Measurable constraint
NFR–01	Performance	The 95th-percentile time to the first visible token shall be 4 seconds or less, and to the complete reply 15 seconds or less for a retrieval answer and 8 seconds or less for a records lookup, measured over 100 requests against the model host. The system shall sustain 10 concurrent conversations with no failed request.
NFR–02	Security	All authentication shall use OpenID Connect with RS256-signed tokens that expire within 15 minutes. All browser traffic shall use TLS 1.2 or higher. The application shall store no password. Of 30 scripted cross-account access attempts and 10 attempts to reach a staff tool from a student session, 0 shall succeed.

NFR-03	Usability	A first-time student shall complete the task "book a study room" in 90 seconds or less and in 4 messages or fewer, with no written instructions. Measured with 5 test users, at least 4 shall succeed.
NFR-04	Reliability and availability	The complete stack shall start from a single `docker compose up` command within 5 minutes on a clean machine. A restart of any container shall lose no reservation and no submitted request. If the model host is unreachable, the system shall return a plain-text service message within 10 seconds rather than waiting indefinitely.
NFR-05	Accuracy	Student supervisor routing accuracy shall be 90 percent or higher on a held-out set of 100 labelled messages. At least 80 percent of the 50 benchmark questions shall be answered correctly with a correct citation, scored by two independent raters and reported separately for English and for Thai.

5. User Stories and Use Case Model

5.1 Core User Stories

ID	User story
US-01	As a **student**, I want to ask about a regulation in chat and receive the answer together with the rule it came from, so that I do not have to search through PDF handbooks or rely on what a senior told me.
US-02	As a **student**, I want to book an advisor meeting, a study room or an instrument room inside the same conversation, so that I do not queue for a slot that someone else already took.
US-03	As a **student**, I want to submit a document request or a library hold and check its state from LINE, Telegram or the website at any hour, so that I do not travel to campus only to ask whether it is ready.
US-04	As a **staff member**, I want to take over a conversation and see everything the student already explained, so that a student whose case I inherit does not have to start again.
US-05	As a **staff member**, I want to ask "show me today's pending requests" and mark one ready without leaving the chat, so that routine administration costs me no context switch.
US-06	As an **advisor or facility owner**, I want each published slot to accept exactly one booking, so that two students never arrive for the same slot.

5.2 Detailed Use Cases

UC-01: Reserve a Bookable Resource

ACTORS

Primary: Student. Secondary: the resource owner, who is an academic advisor, a facility manager or the library. Supporting: Keycloak, PostgreSQL, the Ollama model host, Chatwoot.

PRECONDITIONS

- The student holds an administrator-provisioned account and is signed in through Keycloak.
- The resource owner has published at least one availability window for the resource.
- The application, the database and the model host are reachable.

POSTCONDITIONS

- Success: exactly one reservation row exists for that slot, owned by this student. The slot no longer appears as available. An audit record and a confirmation message exist.
- Failure: no reservation row is created and the slot keeps its previous state.

MAIN FLOW

1. The student writes in the chat that they want a study room on Thursday afternoon.
2. The student supervisor classifies the message as a booking intent and routes it to the Booking agent.
3. The Booking agent reads the student identifier from the verified access token.
4. The agent identifies the resource type from the message and queries free slots for the requested date range.
5. The agent presents up to five options with resource name, day and time.
6. The student selects one option.
7. The agent opens a database transaction, locks the selected slot row, confirms it is still free, writes the reservation and commits.
8. The agent writes an audit record containing actor, action, target and timestamp.
9. The system replies with the confirmed resource, day, time and location, and states the cancellation cut-off.
10. The use case ends.

ALTERNATIVE AND EXCEPTION FLOWS

- **A1. Slot taken during selection (step 7).** The lock shows the slot is no longer free. The transaction rolls back, the agent reports the conflict, re-queries availability and returns to step 5. No reservation is written.
- **A2. No availability in the requested range (step 4).** The agent reports that no slot is free, offers the next available period and returns to step 4 with the new range.
- **A3. Resource type ambiguous (step 4).** The student writes only "book a room". The agent asks which resource type is meant and lists the available types before querying.
- **A4. Ambiguous date (step 4).** The student writes "next week" near a month boundary. The agent states the exact date range it will use and asks the student to confirm before querying.
- **E1. Session expired (step 3).** The access token is invalid. The system does not guess an identity. It asks the student to sign in again and discards the pending action.

- **E2. Overlapping reservation already held (step 7).** The agent refuses the second reservation, shows the existing one and offers to reschedule it instead.
- **E3. Model host unreachable (any step).** The adapter retries twice with a delay. If both retries fail, the system posts a plain-text message within 10 seconds saying the assistant is unavailable, and offers the human handoff of UC-02.
- **E4. Malformed tool arguments from the model (step 7).** The validation layer rejects the call, returns a corrective message to the agent and retries once. On a second failure the agent escalates to a human rather than guessing.
- **E5. Database unavailable (step 7).** The transaction fails and rolls back. The system reports that the booking did not complete and instructs the student to try again. No partial state is written.

UC-02: Answer a Regulation Question and Escalate to a Human

ACTORS

Primary: Student. Secondary: Student services staff member. Supporting: PostgreSQL with pgvector, Chatwoot, the Ollama model host.

PRECONDITIONS

- The corpus is indexed in English and Thai and the similarity threshold is configured.
- At least one staff member is signed in to the staff interface, or an offline queue is available.

POSTCONDITIONS

- Success: the student receives an answer carrying a source citation in the language they used, and the exchange is stored in the conversation history.
- Escalation: the conversation is assigned to a staff member with its full history, and the agent generates no further reply on it.

MAIN FLOW

1. The student asks, on LINE, which documents a visa application requires from the University.
2. Chatwoot delivers the message to the application through a webhook, and the system resolves the sender to a linked student account.
3. The student supervisor classifies the message as a policy question and routes it to the Knowledge agent.
4. The Knowledge agent detects the message language, converts the question into a vector and retrieves the highest-scoring passages from the corpus.
5. The agent confirms that the top passage scores above the similarity threshold.
6. The agent composes an answer using only the retrieved text, in the language of the question.
7. The system returns the answer through Chatwoot to LINE, with the source document title and section.
8. The exchange is written to the conversation history, which is shared across the student's channels.
9. The use case ends.

ALTERNATIVE AND EXCEPTION FLOWS

- **A1. No passage above the threshold (step 5).** The agent does not generate an answer. It states that it does not hold that information and offers to pass the question to a staff member. If the student accepts, the flow continues at A2.
- **A2. Escalation to a human.** The system writes a summary of the conversation, assigns the Chatwoot conversation to an available staff member and marks it human-owned. The agent stops replying. The staff member opens the conversation with the full history visible and answers directly. When the staff member releases the conversation, the agent resumes.
- **A3. Student asks for a human directly (any step).** The flow moves immediately to A2 without attempting an answer.
- **A4. Mixed question (step 3).** The message contains both a policy question and a personal record question. The supervisor routes each part to its agent and returns one combined reply.
- **A5. Thai question with weak Thai generation (step 6).** If the measured Thai generation quality is below the configured threshold, the agent returns the retrieved Thai passage directly with its citation instead of a generated summary, and offers a human handoff.

- **E1. Sender not linked to a student account (step 2).** The system answers public policy questions only and refuses anything personal, with an instruction on how to link the channel account from the signed-in portal.
- **E2. Unsupported language (step 4).** The system replies in English that it supports English and Thai, and offers the human handoff.
- **E3. No staff member available for escalation (step A2).** The conversation enters the pending queue and the student is told that a staff member will reply during office hours. The conversation keeps its full history.
- **E4. Retrieved text contains an instruction aimed at the agent (step 6).** The agent treats all retrieved text as data and never as instruction. Tool access stays limited to the agent's allowlist and no action is taken from document text.

6. Controlled Scope Statement

6.1 In-Scope: Project I Deliverables

1. A public university website with program, admission, announcement and contact pages.
2. An authenticated student portal and a staff console, delivered as one React application with routes gated by Keycloak role.
3. Authentication and role-based access through Keycloak for the roles student, staff, advisor and administrator, with accounts provisioned by the administrator.
4. A student supervisor agent built with LangGraph, routing to five specialist agents.
5. **Knowledge agent:** answers from the indexed English and Thai corpus using retrieval-augmented generation, with a citation on every answer.
6. **Records agent:** read-only access to the signed-in student's own timetable, registration status and fee balance.
7. **Booking agent:** lists, reserves, reschedules and cancels any bookable resource, covering advisor appointments, study rooms, instrument rooms, club activity rooms and workshop seats, with transactional slot locking.
8. **Request agent:** submits and tracks any service request, covering document requests, library holds and maintenance reports, through a four-state lifecycle with notifications.
9. **Catalog agent:** searches the library catalog and reports availability and loan status.
10. A staff supervisor agent with a Queue agent for reading staff queues and an Action agent for state changes and cancellations, with an explicit confirmation step before any action affecting more than one record.
11. Human handoff through Chatwoot, where the agent stands down while a staff member owns the conversation.
12. Three channels: the embedded web widget, LINE and Telegram, sharing one conversation history per student.
13. Local model inference through Ollama behind a provider-abstraction layer, with the model chosen by a measured bake-off.
14. An evaluation harness: 50 benchmark questions in English and Thai, 100 labelled routing messages, and the access-control negative test suite.

15. A Docker Compose deployment that starts the full stack with one command.

6.2 Out-of-Scope: Deferred to Project II or Backlog

1. Online payment processing and payment slip reading. The system records a payment reference only, and a human confirms it.
2. Integration with a real student information system. The project uses its own database with generated data.
3. Grade entry, grade changes and examination result publication.
4. Automated admission decisions and automated scholarship decisions.
5. Course add and drop execution, and prerequisite or timetable-clash checking, which would need a scheduling engine the project does not build.
6. Full-text retrieval inside library holdings, which would require licensed content the project does not hold. The Catalog agent works on metadata only.
7. Any channel beyond the web widget, LINE and Telegram.
8. Cloud deployment with a public domain name, managed TLS certificates and a paid inference provider.
9. Native Android and iOS applications. The web interface is responsive.
10. Voice input, speech recognition, and languages beyond English and Thai.
11. Training or fine-tuning a language model. The project runs an existing open-weight model.
12. Learning-management functions such as assignments, quizzes and course content delivery.
13. Advanced analytics dashboards, reporting exports and multi-institution scaling.

6.3 Constraints, Assumptions and Limitations

Constraints

- **Time.** Ten working weeks for Project I, with a four-person team that carries no other project this semester. W10 holds no new functionality.
- **Budget.** Zero. Every component must be free or open source, which excludes any paid model API, managed database or paid hosting for the prototype.
- **Hardware.** Four laptops, two with 16 GB of memory and two with 8 GB, and no graphics processor. This caps both the size of the local model and the number of services any one machine runs.
- **Platform.** Docker Compose on a single host, with the model served from a second host on the local network. No orchestration platform and no horizontal scaling.

Assumptions

- An open-weight model that runs within 16 GB of unified memory is reliable enough at tool calling for this agent design. The Week 2 bake-off verifies this rather than assuming it, and risk R1 names the fallback.
- A corpus authored from publicly published university handbooks is a fair proxy for a real institutional corpus in structure, length and ambiguity.
- Students use a chat interface without training, because they already use LINE and similar applications daily.
- Resource owners are willing to publish availability in a digital calendar rather than on a printed sheet.
- The four team members can reach the model host over a shared network, or over a free mesh network service if the campus network blocks it.

- The interfaces frozen in W1 remain stable, so the four parallel tracks do not block one another.

Limitations

- The prototype runs on team hardware, so it is not continuously available and the demonstration depends on two specific machines.
- Response times reflect a local model on consumer hardware and will be slower than a hosted service. A production deployment would need a graphics processor or a paid provider, as stated in Section 2.1.
- All student records, resources and catalog entries are generated, so the project demonstrates correct behaviour but not real-world data quality problems such as duplicate or incomplete records.
- Thai quality is expected to be lower than English quality. The report states the measured gap rather than hiding it.
- Usability is measured with five test users, which is enough to find major problems but is not a statistically representative sample.
- The evaluation benchmark is written by the team, so it reflects the team's view of which questions matter. Two independent raters score the answers to reduce, but not remove, that bias.